Database Model Design

Model Design

The database is designed around three goals:

1. **Thread-centric discussion**
    - Discussions are organized by `Thread`, with `Post` as the content unit.
    - `Post.parentId` enables reply trees (nested comments).
    - `Thread.mainPostId` ensures each thread has a clear root post.
2. **Sports-aware structure**
    - `Team` and `Match` are separate core entities.
    - `Match` links two teams (`homeTeamId`, `awayTeamId`) and game metadata (date, scores, status).
    - A match can have one dedicated discussion thread (`Thread.matchId` unique), which supports match-specific conversations.
3. **Platform/community features**
    - `Follow` models user-to-user social graph.
    - `Poll`, `PollOption`, `PollVote` support voting features in threads/posts.
    - `Tag` + `TagThread` provide many-to-many thread categorization.
    - `Report` and `Appeal` support moderation workflows.
    - `Sentiment` stores one aggregated sentiment record per match (`matchId` unique).

Integrity and constraints in the design

- Unique user identity: `User.email`, `User.username`.
- No duplicate matches: `@@unique([homeTeamId, awayTeamId, date])` on `Match`.
- One dedicated thread per match: `Thread.matchId @unique`.
- One sentiment record per match: `Sentiment.matchId @unique`.
- No duplicate follow edges: `@@unique([followerId, followingId])`.
- No duplicate tag assignment per thread: `@@unique([tagId, threadId])`.

This design balances normalization (separate entities for users, teams, matches, posts) with practical product constraints (unique links and moderation/social support).

Overview

This database is designed for a sports discussion platform. It supports user accounts, teams, matches, match/general threads, post replies, polls, moderation, follow relationships, and sentiment analysis. The design is centered on relational integrity using foreign keys and unique constraints.

Models, Key Fields, and Relationships

`User`

- Key fields: `id`, `email` (unique), `username` (unique), `role`, `isBanned`, `themeMode`, `favoriteTeamId`.
- Relationships:
  - One user can create many `Post`, `Thread`, `Poll`, `Report`, `Appeal`, and `PollVote` records.
  - Optional favorite team (`favoriteTeamId -> Team.id`).
  - User-to-user follow graph through `Follow`.

`Team`

- Key fields: `id`, `name`, `logoUrl`, `wins`, `losses`, `conference`, `division`.
- Relationships:
  - One team can appear in many matches as home or away.
  - One team can be linked to many threads.
  - One team can be selected as favorite by many users.

`Match`

- Key fields: `id`, `homeTeamId`, `awayTeamId`, `date`, `status`, `homeScore`, `awayScore`, `endedAt`.
- Constraint: `@@unique([homeTeamId, awayTeamId, date])` to avoid duplicate match rows.
- Relationships:
  - Belongs to one home team and one away team.
  - Can be linked to a dedicated thread (via `Thread.matchId`).
  - Has at most one sentiment record (`Sentiment.matchId` is unique).

`Thread`

- Key fields: `id`, `title`, `mainPostId` (unique), `createdById`, optional `teamId`, optional `matchId` (unique), `isVisible`, `isClosed`, `opensAt`, `closesAt`.
- Relationships:
  - Created by one user.
  - Can belong to one team and/or one match.
  - Contains many posts, polls, reports, and tags.
  - `mainPostId` points to the thread's root post.

`Post`

- Key fields: `id`, `content`, `threadId`, `authorId`, `parentId`, `version`, `isVisible`.
- Relationships:
  - Belongs to one author (`User`).
  - Optionally belongs to one thread.
  - Supports nested replies through self-reference (`parentId`).
  - Can be reported.
  - Can optionally have one attached poll.

`Poll`, `PollOption`, `PollVote`

- `Poll`: question, optional thread/post attachment, creator, deadline.
- `PollOption`: options belonging to a poll.
- `PollVote`: links a user to a selected option in a poll.
- Relationships:

    - One poll has many options and many votes.
    - Each vote belongs to one poll option and one user.

`Tag` and `TagThread`

- `Tag` stores unique tag names.
- `TagThread` is a join table for many-to-many thread-tag mapping.
- Constraint: `@@unique([tagId, threadId])`.

`Follow`

- Key fields: `followerId`, `followingId`.
- Purpose: stores user-to-user following.
- Constraint: `@@unique([followerId, followingId])` to prevent duplicate follow edges.

`Report`

- Key fields: `reportedById`, optional `postId`, optional `threadId`, `reason`, moderation fields (`aiVerdict`, `toxicity`, `isResolved`).
- Purpose: moderation queue for reported content.

`Appeal`

- Key fields: `userId`, `reason`, `status`, `reviewedAt`.
- Purpose: stores ban appeals and review results.

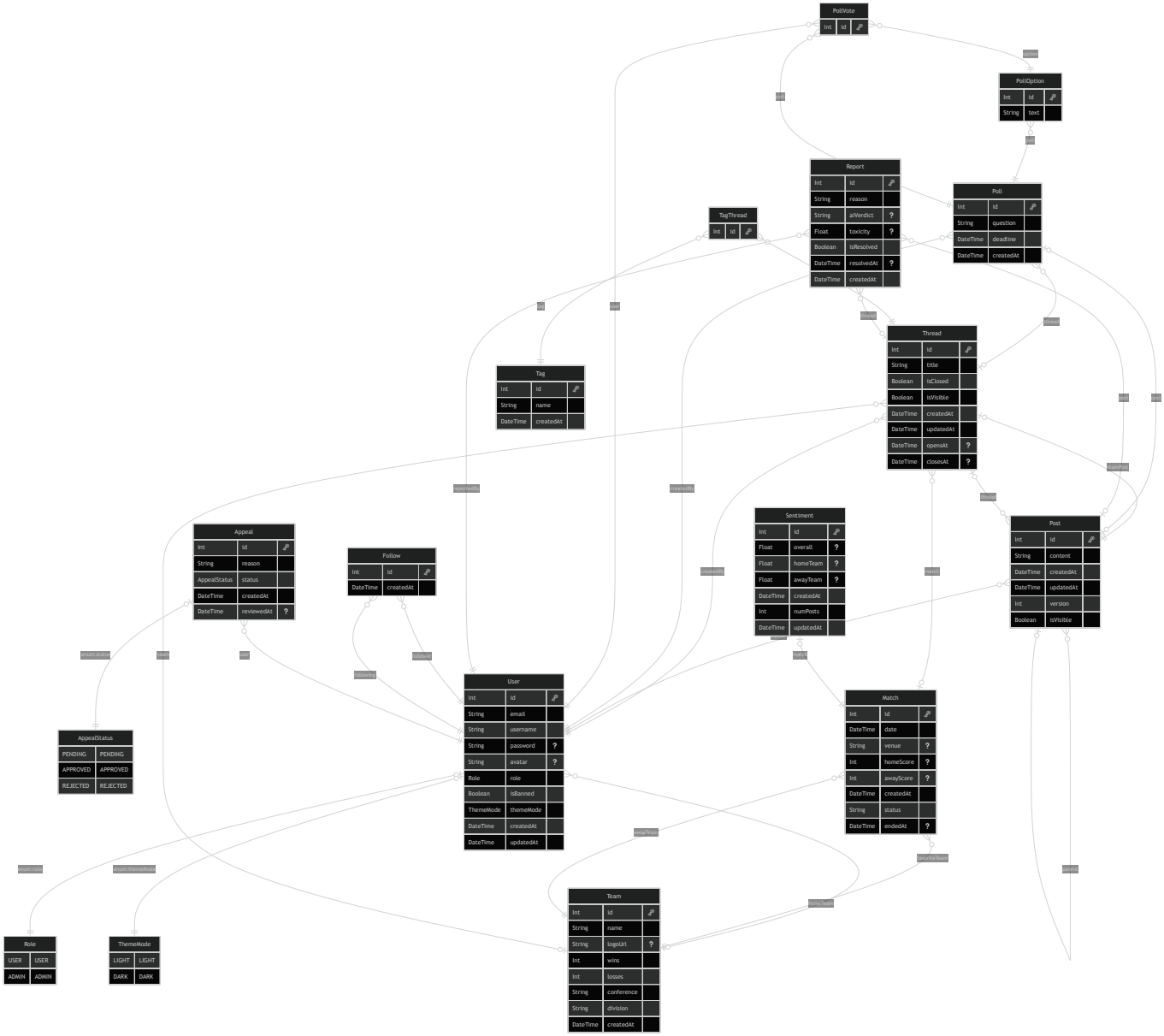
`Sentiment`

- Key fields: `matchId` (unique), `overall`, `homeTeam`, `awayTeam`, `numPosts`.
- Purpose: stores aggregated sentiment for a match discussion.

Relationship Summary

- User 1-to-many Post/Thread/Poll/Report/Appeal/PollVote.
- Team 1-to-many Match (home and away roles).
- Match to Thread is constrained to one dedicated thread by unique `Thread.matchId`.
- Match to Sentiment is one-to-one by unique `Sentiment.matchId`.
- Thread 1-to-many Post/Poll/Report.
- Thread many-to-many Tag via `TagThread`.
- User many-to-many User via `Follow`.

ERD Diagram


PollVote
Int id
PollOption
Int id
String text
Report
Int id
String reason
String aiVerdict ?
Float toxicity ?
Boolean isResolved
DateTime resolvedAt ?
DateTime createdAt
Poll
Int id
String question
DateTime deadline
DateTime createdAt
TagThread
Int id
Tag
Int id
String name
DateTime createdAt
Thread
Int id
String title
Boolean isClosed
Boolean isVisible
DateTime createdAt
DateTime updatedAt
DateTime opensAt ?
DateTime closesAt ?
Appeal
Int id
String reason
AppealStatus status
DateTime createdAt
DateTime reviewedAt ?
Follow
Int id
DateTime createdAt
Sentiment
Int id
Float overall ?
Float homeTeam ?
Float awayTeam ?
DateTime createdAt
Int numPosts
DateTime updatedAt
Post
Int id
String content
DateTime createdAt
DateTime updatedAt
Int version
Boolean isVisible
AppealStatus
PENDING PENDING
APPROVED APPROVED
REJECTED REJECTED
User
Int id
String email
String username
String password ?
String avatar ?
Role role
Boolean isBanned
ThemeMode themeMode
DateTime createdAt
DateTime updatedAt
Match
Int id
DateTime date
String venue ?
Int homeScore ?
Int awayScore ?
DateTime createdAt
String status
DateTime endedAt ?
Team
Int id
String name
String logoUrl ?
Int wins
Int losses
String conference
String division
DateTime createdAt
Role
USER USER
ADMIN ADMIN
ThemeMode
LIGHT LIGHT
DARK DARK